

南開大學
Nankai University

计算机学院

并行程序设计 **Lab2** 报告

SIMD 编程实验

姓名：赵宇哲

学号：2411094

专业：计算机科学与技术

2026 年 5 月 8 日

项目仓库：https://github.com/Aurora1094/NKU-ParallelProgramming

摘 要

本实验选择近似最近邻搜索（Approximate Nearest Neighbor Search, ANN）作为 SIMD 并行化选题，围绕 DEEP100K 数据集上高维向量查询中的距离计算与候选筛选瓶颈，在 ARM aarch64 平台上实现并比较了 Flat-SIMD、SQ-SIMD 和 PQ-ADC 三条优化路线。

首先，本文以原始 Flat Scan 作为基线，逐步实现 NEON SIMD 内积、循环展开、多累加器和 prefetch 优化。实验结果表明，Flat-SIMD 能够在不损失 `recall@100` 的情况下显著降低精确扫描开销，其中 `v4` 将平均查询延迟从 `v0` 的 15955.27 μs 降低至 5214.38 μs。

在此基础上，本文进一步引入标量量化（Scalar Quantization, SQ），通过量化偏移修正项恢复粗排排序质量，并使用 uint8 NEON SIMD 加速粗排阶段。最终版本 `v5_4_p100` 采用 corrected SQ 粗排、uint8 SIMD 粗排和 float SIMD 精排，在保持 `recall@100=0.99995` 的同时，将平均查询延迟降低至 3675.09 μs，相对原始基线获得约 4.34× 加速。

此外，本文实现了 PQ-ADC 与 LUT SIMD 分支，用于分析不同召回率目标下的 latency-recall 权衡；实验同时尝试了 OPQ 和 OpenMP，但二者均未作为最终主线。

实验结果表明，当应用允许 `recall@100` 处于 0.9–0.95 区间时，PQ-ADC + rerank 可以以一定召回损失换取更低延迟；而在接近 1.0 的高召回目标下，`v5_4_p100` 是本文的最佳综合方案。最后，本文结合 `perf stat`、cache 事件、汇编指令验证、自动向量化对比和 x86 平台补充实验，从 cycles、instructions、branch-misses、访存行为和底层 SIMD 指令等角度解释了不同版本的性能差异。

关键词：近似最近邻搜索；SIMD；ARM NEON；标量量化；乘积量化；性能剖析

目录

1 前言 **1**
1.1 问题概述 . . . 1
1.2 实验平台 . . . 1
1.3 数据集与查询任务 . . . 1
1.4 评价指标 . . . 2
1.5 测试方法 . . . 2

2 算法设计与实现 **2**
2.1 ANN 查询流程与优化思路 . . . 2
2.2 Flat-SIMD . . . 3
2.3 SQ（Scalar Quantization） . . . 3
2.4 PQ（Product Quantization） . . . 4
2.5 混合方法（SQ/PQ + rerank） . . . 6

3 性能测试 **7**
3.1 主线版本演进（`v0`–`v5_5`） . . . 7
3.2 Flat-SIMD 消融（`v1`–`v4`） . . . 8
3.3 SQ 路线分析（`v5_1`–`v5_5`） . . . 8
3.4 PQ 路线分析（`v6_1`–`v6_2`） . . . 10

4 Profiling 与汇编分析 **11**
4.1 `perf stat` 与 `cache` 分析 . . . 11
4.2 汇编指令验证 . . . 12
4.3 自动向量化与手写 SIMD 对比 . . . 13
4.4 ARM 与 x86 自动向量化对比 . . . 14

5 结果分析与讨论 **15**
5.1 实验分析总结 . . . 15
5.2 实验局限讨论 . . . 15

1 前言

1.1 问题概述

近似最近邻搜索（Approximate Nearest Neighbor Search, ANN）的目标是在高维向量集合中快速找到与查询向量最相近的若干向量，常用于图像检索、语义检索、推荐系统和大模型检索增强生成等场景。ANN 的核心问题是在较低查询延迟下保持较高召回率。

本实验围绕 DEEP100K 数据集上的 ANN 查询任务，具体研究 Flat Scan 中大量 96 维内积计算带来的性能瓶颈。本文分别实现 Flat-SIMD、SQ-SIMD 和 PQ-ADC 三条优化路线，利用 ARM NEON SIMD 与量化粗排降低查询延迟，并结合 `recall@100`、`perf stat`、cache 事件和汇编指令分析不同优化版本的性能差异。

1.2 实验平台

lab2 中，主实验在课程提供的 ARM aarch64 集群环境中完成，主要使用 ARM NEON 指令进行手写 SIMD 优化；补充实验在本地 x86 平台的 WSL 环境中完成，用于比较普通 C++ 自动向量化在不同平台上的表现。[1]。

表 1: 实验平台与软件环境

项目	ARM 平台	x86 平台
操作系统	openEuler 22.03 LTS SP4	WSL Ubuntu 22.04
CPU 架构	aarch64	x86_64
CPU 型号	HiSilicon Kunpeng-920	Intel Core i9-14900HX
CPU 数量	8	32 logical CPUs
SIMD	ARM NEON / ASIMD, 128-bit	AVX, 32-byte vectors observed
编译器	g++ 10.3.1	g++ 11.4.0
编译参数	`-O2 -fopenmp -lpthread -std=c++11`	`-O2/-O3/-O3 -march=native`

其中，ARM 为本文主要性能测试和最终结果所使用的平台；x86 只用于自动向量化的跨平台补充实验，不作为最终提交版本的运行平台。

1.3 数据集与查询任务

本文使用 DEEP100K 数据集：base 文件包含 100000 个 96 维 float32 向量，query 文件包含 10000 个 96 维 float32 查询向量，ground truth 文件提供每个 query 的 top100 真实近邻。主实验使用前 2000 个 query 进行计时和 recall@100 统计。

实验任务为：对于每个 query，在 100000 个 base 向量中检索最相似的候选向量，并与 ground truth 中的 top100 结果进行比较。原始 Flat Scan 方法直接扫描全部 base 向量，可以视为高召回但高延迟的基线；SQ 和 PQ 路线则通过量化粗排与候选精排，在降低查询延迟的同时尽量保持较高召回率。

[1]由于 ARM 集群和本地 x86 平台的 CPU 型号、频率、缓存、内存系统和编译器后端均不同，本文不将二者的绝对 latency 作为公平性能比较，只用于分析不同 ISA 后端下自动向量化代码生成的差异。

1.4 评价指标

本文使用 average latency、recall@100 和 speedup 三个指标。Average latency 表示平均每个 query 的查询时间；recall@100 表示返回结果与 ground truth top100 的重合比例；speedup 以 `v0` 为 baseline：

$$\text{Speedup}(v) = T_{v0}/T_v. \tag{1}$$

1.5 测试方法

所有主实验均使用相同数据集、查询任务和评价指标，在 ARM aarch64 平台上以 `g++ main.cc -o main -O2 -fopenmp -lpthread -std=c++11` 编译运行。不同版本通过切换 `main.cc` 中的头文件和查询函数实现，最终提交版本为 `v5_4_p100`（`rerank_p=100`）。主性能结果采用多次复测平均值；`perf stat` 和汇编输出仅用于硬件事件分析与指令验证。

2 算法设计与实现

2.1 ANN 查询流程与优化思路

本文围绕“加速距离计算”和“减少精排候选数量”两类思路展开优化，如图 1 所示。Flat-SIMD 路线直接优化原始 float 内积；SQ-SIMD 路线先用 uint8 量化向量进行粗排，再对 Top-p 候选做 float 精排；PQ-ADC 路线则通过乘积量化和 LUT 查表实现近似粗排，用于分析 latency-recall 权衡。

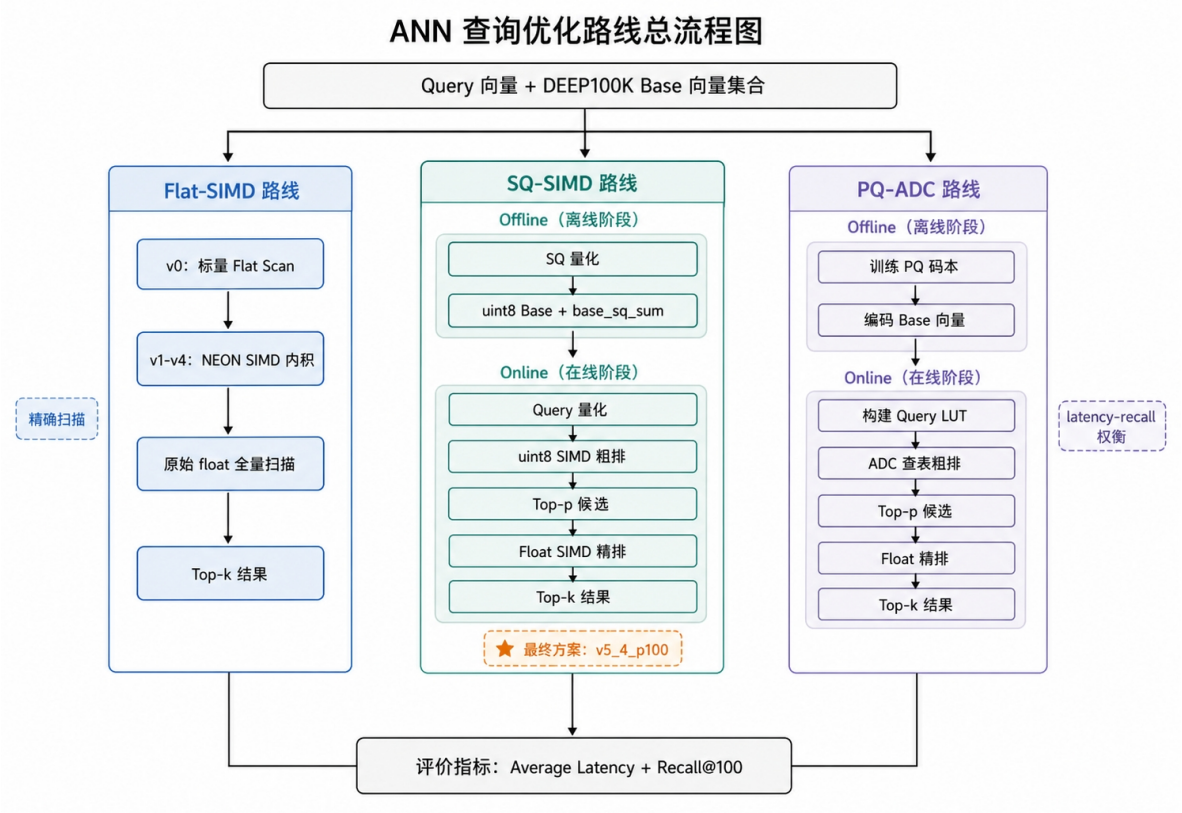


图 1: ANN 查询优化路线总流程图

后续小节将分别介绍 Flat-SIMD、SQ-SIMD 和 PQ-ADC 三条路线的实现。

2.2 Flat-SIMD

原理： SIMD（单指令多数据）通过一条指令同时处理多个数据元素，从而加速向量计算。在 ANN 查询中，主要计算开销来自查询向量与大量 base 向量之间的内积计算，因此适合使用 ARM NEON SIMD 进行优化。

Algorithm 1 Flat-SIMD 查询流程

Input: query 向量 q，base 向量集合 $X = \{x_1, x_2, \ldots, x_N\}$，返回结果数 k
Output: top-k 查询结果

初始化 top-k 候选集合 H
for $i = 1$ to N **do**
  $sum \leftarrow 0$
  for $j = 0$ to $d - 1$ step 4 **do**
    使用 NEON 加载 $q[j : j + 3]$ 和 $x_i[j : j + 3]$
    使用 SIMD 乘加累加局部内积
  end for
  对 SIMD 累加器进行水平求和，得到 $score(q, x_i)$
  用 $score(q, x_i)$ 更新 top-k 候选集合 H
end for
return H

数学推导：

- **内积相似度：** 对于查询向量 q 和 base 向量 x_i，本文使用内积作为相似度度量：

$$score(q, x_i) = \sum_{j=0}^{d-1} q_j x_{i,j}.$$

- **SIMD 分块计算：** DEEP100K 中向量维度为 $d = 96$。ARM NEON 一次可以处理 4 个 float，因此可将内积分解为：

$$score(q, x_i) = \sum_{t=0}^{23} \sum_{r=0}^{3} q_{4t+r} x_{i,4t+r}.$$

- **复杂度：** Flat Scan 每个 query 需扫描全部 N 个 d 维 base 向量，距离计算复杂度为 $O(Nd)$；若维护 top-k，总复杂度可写为

$$O(Nd + N \log k).$$

  SIMD 不改变渐进复杂度，只降低内层乘加的常数开销。

优势： SIMD 不改变原始 float 内积结果，可以在保持 recall 不变的情况下显著降低距离计算开销。

缺点： Flat-SIMD 仍需完整扫描全部 base 向量，只能加速单次距离计算，不能减少参与计算的向量数量。

2.3 SQ（Scalar Quantization）

原理： SQ（标量量化）将每个维度的 float 数据映射为低精度整数，例如 8 位无符号整数，从而减少存储开销并加速粗排阶段计算。

Algorithm 2 SQ-SIMD 粗排与精排流程

Input: query 向量 q，量化 base 向量 X_q，原始 base 向量 X，修正项 $base_sq_sum$，候选数 p，返回结果数 k
Output: top-k 查询结果
1: 将 query 向量 q 量化为 uint8 向量 q_q
2: 初始化 Top-p 粗排候选集合 C_p
3: **for** $i = 1$ to N **do**
4: 使用 uint8 NEON SIMD 计算 $dot(q_q, x_{q,i})$
5: 根据 $base_sq_sum_i$ 修正粗排分数
6: 用修正后的粗排分数更新 Top-p 候选集合 C_p
7: **end for**
8: 初始化 top-k 精排结果集合 H
9: **for** 每个候选 $x_i \in C_p$ **do**
10: 使用 float NEON SIMD 重新计算原始内积 $score(q, x_i)$
11: 用精排分数更新 top-k 结果集合 H
12: **end for**
13: **return** H

数学推导：

- **量化映射：**对于 float 值 x，将其映射到 $[0, 255]$ 范围：

$$x_q = \text{round}\left(\frac{x - min}{scale}\right), \qquad x \approx min + scale \cdot x_q.$$

- **量化内积修正：**若直接使用 uint8 点积：

$$score_q(q, x_i) = \sum_j q_{q,j} x_{q,i,j},$$

会忽略量化偏移项。因此，本文预计算每个 base 向量的修正项：

$$base_sq_sum_i = \sum_j x_{q,i,j}.$$

- **复杂度：** SQ 粗排扫描全部 N 个量化向量，复杂度为 $O(Nd)$；对 Top-p 候选进行 float 精排的复杂度为 $O(pd)$。若考虑候选维护，总复杂度为

$$O(Nd + N \log p + pd).$$

其优势主要来自 uint8 数据宽度更小，以及精确 float 计算只保留在 p 个候选上。

优势：SQ 降低了粗排阶段的数据宽度，结合 SIMD 后可以提高粗排吞吐量；通过修正项和 rerank 可以保持较高 recall。

缺点： SQ 属于近似表示，若不修正量化偏移或候选数过小，会导致 recall 明显下降。

2.4 PQ（Product Quantization）

原理： PQ（乘积量化）将高维向量划分为多个低维子空间，并在每个子空间中分别训练聚类中心。base 向量用各子空间中心编号表示，查询时通过查表累加快速得到近似分数。

Algorithm 3 PQ-ADC 查询流程

Input: query 向量 q，PQ codebook $C^{(m)}$，base 编码 $code_i^{(m)}$，候选数 p，返回结果数 k
Output: top-k 查询结果

```
将 query 向量划分为 M 个子向量 q^(1), q^(2), ..., q^(M)
for m = 1 to M do
    for r = 1 to K_s do
        计算 LUT[m][r] = ⟨q^(m), c_r^(m)⟩
    end for
end for
初始化 Top-p 粗排候选集合 C_p
for i = 1 to N do
    score_PQ ← 0
    for m = 1 to M do
        score_PQ ← score_PQ + LUT[m][code_i^(m)]
    end for
    用 score_PQ 更新 Top-p 候选集合 C_p
end for
对 C_p 中候选使用原始 float SIMD 内积重新精排
return 精排后的 top-k 结果
```

数学推导：

- **子空间划分：**将 d 维向量划分为 M 个子空间：

$$x = [x^{(1)}, x^{(2)}, \ldots, x^{(M)}].$$

- **子空间编码：**每个子空间训练 codebook：

$$C^{(m)} = \{c_1^{(m)}, c_2^{(m)}, \ldots, c_{K_s}^{(m)}\}.$$

对 base 向量进行编码：

$$code_i^{(m)} = \arg\min_r \|x_i^{(m)} - c_r^{(m)}\|^2.$$

- **ADC 查表计算：**查询时构建查找表：

$$LUT[m][r] = \langle q^{(m)}, c_r^{(m)} \rangle.$$

则 PQ 近似分数为：

$$score_{PQ}(q, x_i) = \sum_{m=1}^{M} LUT[m][code_i^{(m)}].$$

- **复杂度：** LUT 构建复杂度约为 $O(K_s d)$；ADC 粗排对每个 base 进行 M 次查表累加，复杂度为 $O(NM)$。加上 Top-p 维护与精排，总复杂度为

$$O(K_s d + NM + N \log p + pd).$$

通常 $M \ll d$，因此 PQ 粗排计算量低于完整 float 内积扫描。

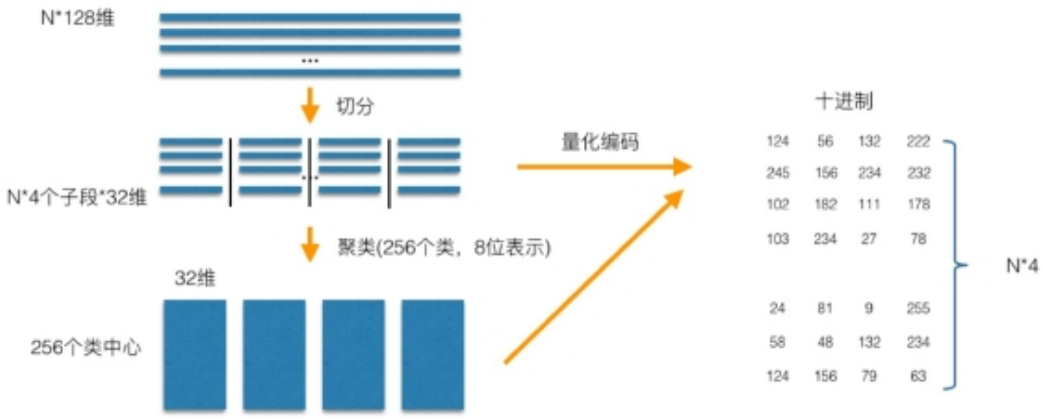


图 2: PQ 乘积量化原理示意图

2.5 混合方法（SQ/PQ + rerank）

原理：混合方法先使用 SQ 或 PQ 进行快速粗排，筛选出候选集合，再使用原始 float 向量进行精排，以兼顾查询速度和召回率。

Algorithm 4 SQ/PQ + rerank 混合查询流程

Input: query 向量 q，base 向量集合 X，粗排方法 $score_{approx}$，候选数 p，返回结果数 k
Output: top-k 查询结果

初始化 Top-p 粗排候选集合 C_p
for $i = 1$ to N **do**
    使用 SQ-SIMD 或 PQ-ADC 计算近似分数 $score_{approx}(q, x_i)$
    用近似分数更新 Top-p 候选集合 C_p
end for
初始化 top-k 精排结果集合 H
for 每个候选 $x_i \in C_p$ **do**
    使用原始 float NEON SIMD 计算精确内积 $score(q, x_i)$
    用精确分数更新 top-k 结果集合 H
end for
return H

数学推导：

- **粗排候选生成：**

$$C_p(q) = \mathrm{TopP}(score_{approx}(q, x_i)).$$

- **精排计算：**对候选集合重新计算原始 float 内积：

$$score(q, x_i) = \sum_{j=0}^{d-1} q_j x_{i,j}, \qquad x_i \in C_p(q).$$

- **复杂度：**混合方法可概括为

$$O(T_{approx} + N \log p + pd),$$

其中 T_{approx} 表示粗排代价。SQ 中 $T_{approx} = O(Nd)$，PQ-ADC 中 $T_{approx} = O(K_s d + NM)$。因此 p 直接影响精排延迟和 recall。

优势：粗排减少进入精确计算的候选数量，精排保证最终结果质量，是本文高 recall 目标下的主要优化路线。

缺点：候选数 p 需要权衡。p 过小会损失 recall，p 过大会增加 rerank 延迟。

3 性能测试

3.1 主线版本演进（v0–v5_5）

为观察不同优化策略对整体查询性能的影响，本文首先对主线版本进行统一性能对比。图 3 给出了各版本的平均查询延迟，图 4 给出了相对于原始 baseline v0 的加速比。所有版本均在相同 DEEP100K 数据集、相同查询任务和相同评价指标下测试。

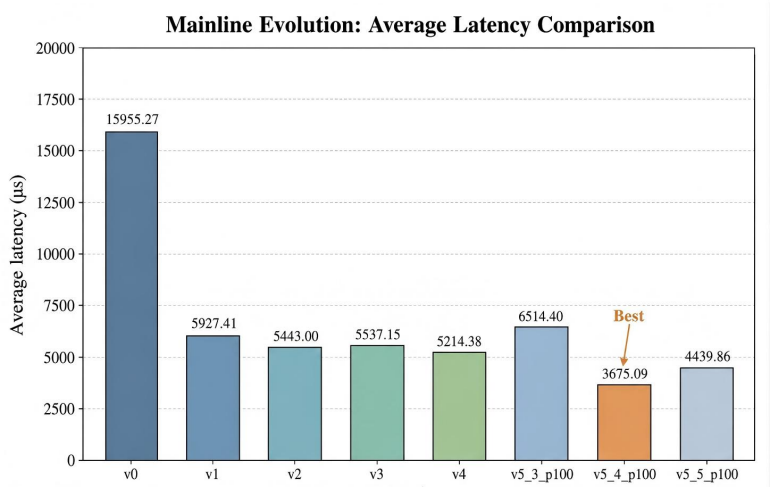


图 3: 主线演进版本的平均延迟对比

从图 3 可以看到，原始 Flat Scan 版本 v0 的平均延迟为 15955.27 μs。加入基础 NEON SIMD 后，v1 降至 5927.41 μs，说明 96 维 float 内积是主要瓶颈；随后 v2 通过循环展开和多累加器继续降低延迟，v4 结合 prefetch 后达到 5214.38 μs，成为 Flat-SIMD 路线最佳版本。

在 SQ 路线中，v5_3_p100 通过量化偏移修正保持了 recall@100=0.99995，但由于粗排仍为标量 uint8 计算，延迟为 6514.40 μs，慢于 v4。进一步将 uint8 dot product 改为 NEON SIMD 后，v5_4_p100 延迟降至 3675.09 μs，同时保持 recall@100=0.99995，成为本文高召回目标下的最优主版本。v5_5_p100 尝试使用 nth_element，但延迟为 4439.86 μs，没有超过 v5_4_p100，因此作为负向消融结果保留。

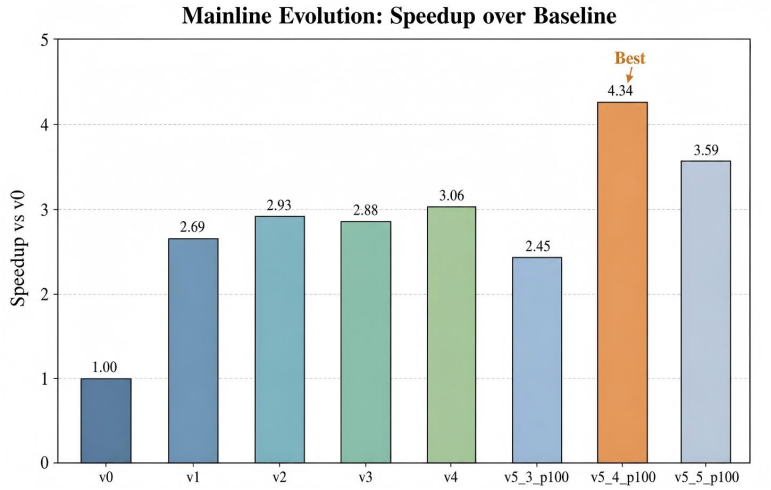


图 4: 主线演进版本的加速比对比

从图 4 可以看到，v1 相对 v0 获得约 2.69× 加速，v4 提升到约 3.06×，最终版本 v5_4_p100 达到约 4.34×。这说明本文的最优结果不是来自单一 SIMD 替换，而是来自 corrected SQ 粗排、uint8 SIMD 粗排和 float SIMD 精排的组合优化。

3.2 Flat-SIMD 消融（v1–v4）

为进一步观察 Flat-SIMD 路线中各项优化的独立作用，本文对 v0 到 v4 进行了消融对比，结果如图 5 所示。

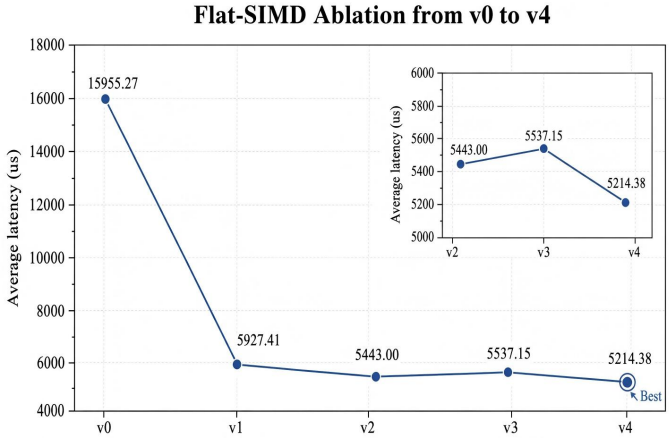


图 5: Flat-SIMD 路线的消融实验结果

可以看到，v1 相比原始 v0 带来了最大的单步性能提升，平均延迟从 15955.27 μs 降至 5927.41 μs，说明 96 维 float 内积计算是 Flat Scan 的主要瓶颈。随后，v2 通过循环展开和多累加器将延迟进一步降低到 5443.00 μs，说明减少循环控制开销和累加依赖链仍能带来收益。

v3 的平均延迟为 5537.15 μs，略慢于 v2，说明末尾水平求和并不是该阶段的主要瓶颈。v4 在 v2 基础上加入 prefetch 后降至 5214.38 μs，成为 Flat-SIMD 路线最佳版本。总体来看，Flat-SIMD 能有效降低原始 float 内积开销，但仍需完整扫描全部 base 向量，因此优化上限受到全量扫描模式限制。

3.3 SQ 路线分析（v5_1–v5_5）

SQ 路线是本文最终主方案所在的优化分支，其核心思想是先使用量化后的 uint8 向量进行快速粗排，再对少量候选使用原始 float 向量精排。图 6 给出了 SQ 路线关键版本的消融结果。

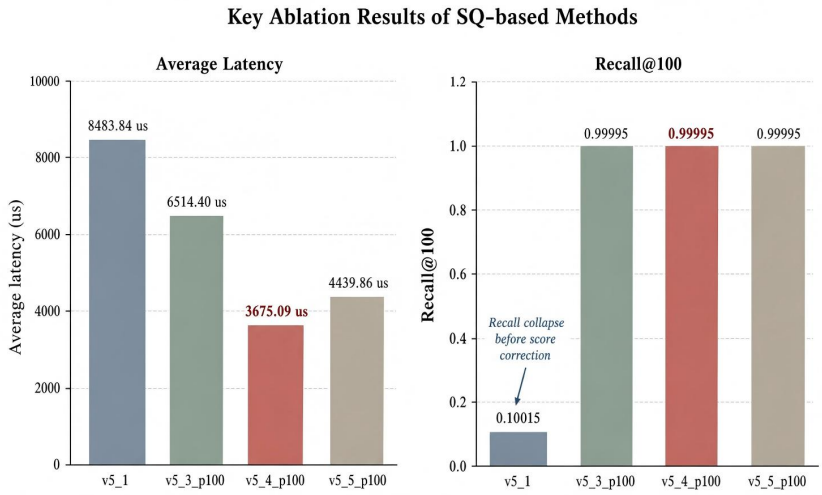


图 6: SQ 路线关键版本的消融实验结果

从图 6 可以看到，v5_1 虽然引入了 SQ 粗排，但 recall@100 仅为 0.10015，说明直接使用 uint8 dot product 会破坏原始 float 内积的排序关系。加入量化偏移修正后，v5_3_p100 将 recall@100 恢复到 0.99995，但延迟仍为 6514.40 μs，说明标量 SQ 粗排本身仍然较慢。

v5_4_p100 进一步使用 uint8 NEON SIMD 加速粗排，平均延迟降至 3675.09 μs，同时保持 recall@100=0.99995，成为 SQ 路线中综合表现最好的版本。v5_5_p100 尝试使用 nth_element 进行候选选择，但延迟为 4439.86 μs，没有超过 v5_4_p100。

为进一步观察 SIMD 粗排的作用，图 7 对比了 v5_3 与 v5_4 在不同 rerank_p 下的延迟。

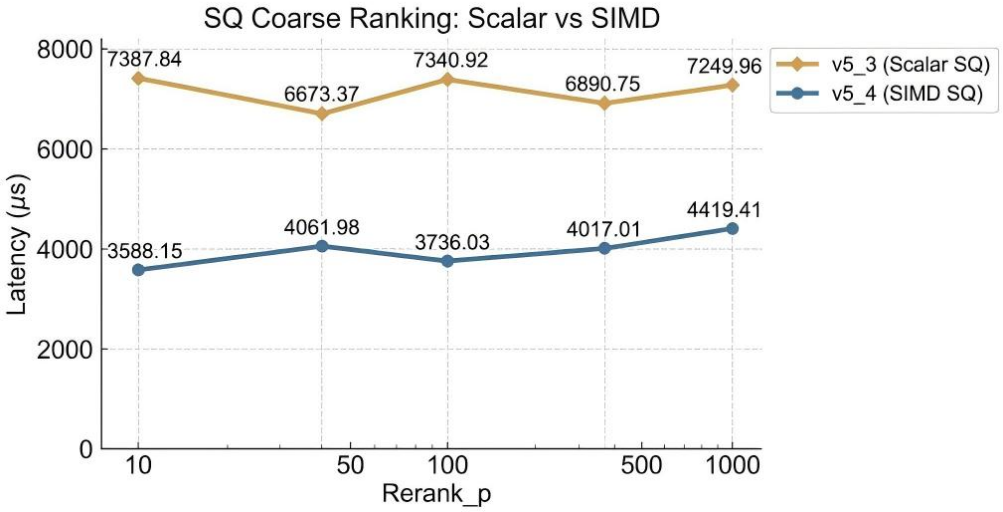


图 7: SQ 路线中标量粗排与 SIMD 粗排的延迟对比

可以看到，在所有测试的 rerank_p 下，v5_4 均明显快于 v5_3。两者的主要区别在于粗排阶段：v5_3 使用标量 uint8 dot product，而 v5_4 使用 uint8 NEON SIMD 粗排。因此，SQ 路线真正产生性能优势的关键不是量化本身，而是量化后能够使用更高并行度的 uint8 SIMD 计算。

图 8 展示了 v5_4 在不同 rerank_p 下的 latency-recall 权衡关系。

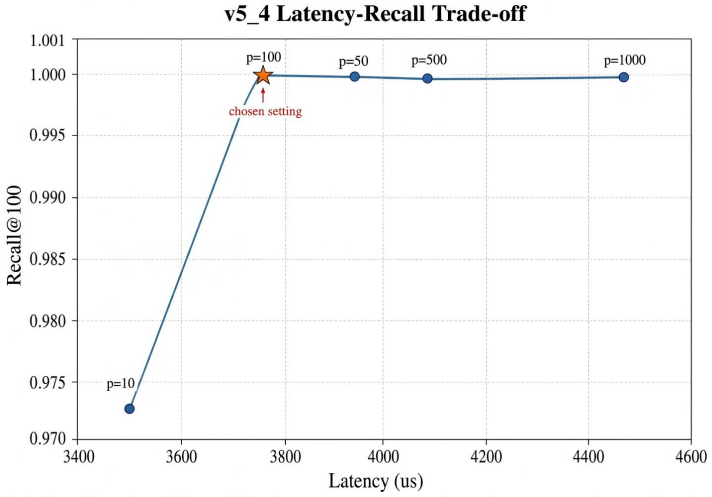


图 8: v5_4 在不同 rerank_p 下的 latency-recall 权衡

当 rerank_p=10 时，延迟最低，但 recall@100 下降到约 0.973，说明候选集合过小会使部分真实近邻无法进入精排阶段。当 rerank_p 增大到 50、100、500 和 1000 后，recall@100 基本恢复到接近

1.0。其中，`p=100` 在保持 0.99995 `recall@100` 的同时具有较低延迟，因此本文最终选择 `v5_4_p100` 作为主版本。

综上，SQ 路线的关键结论是：naive SQ 会导致召回率崩溃，corrected SQ 可以恢复召回率，而 uint8 NEON SIMD 粗排是降低延迟的主要来源。在高 recall 目标下，`v5_4_p100` 是本文最优的 SQ 方案。

从体系结构角度看，SQ-SIMD 相比 Flat-SIMD 的优势不仅来自整数 SIMD 指令本身，也来自数据宽度和访存量的降低。DEEP100K 中每个 query 需要扫描 100000 个 96 维 base 向量。若使用原始 float32 表示，单个 query 至少需要读取

$$100000 \times 96 \times 4\text{B} \approx 36.6\text{MiB}$$

的 base 向量数据；而 SQ 粗排阶段使用 uint8 表示时，对应主扫描数据量约为

$$100000 \times 96 \times 1\text{B} \approx 9.15\text{MiB},$$

仅为 float32 扫描的四分之一。

不过，在线查询还包含 query 量化、`base_sq_sum` 修正、top-p 候选维护以及 float SIMD rerank 等额外开销。这也解释了为什么 `v5_4_p100` 明显快于 `v4`，但没有达到理论数据宽度比例对应的极限加速。

3.4 PQ 路线分析（`v6_1`–`v6_2`）

PQ 路线用于探索另一类近似搜索策略。与 SQ 路线相比，PQ 通过子空间量化和查表式 ADC 计算进一步降低粗排成本，但也会引入更明显的近似误差。因此，本节主要关注其 latency-recall 权衡，而不是将其简单视为成功或失败。

图 9 展示了 PQ 路线的 latency-recall 权衡关系。随着 `rerank_p` 增大，更多候选进入 float 精排阶段，recall@100 从 `v6_1_p100` 的 0.4414 逐步提升到 `p=5000` 时的约 0.9905，但延迟也从 673.67 μs 增加到约 3900 μs。因此，PQ 分支适合描述“低延迟但可牺牲 recall”的近似搜索路线，而不是本文高 recall 目标下的最终主版本。

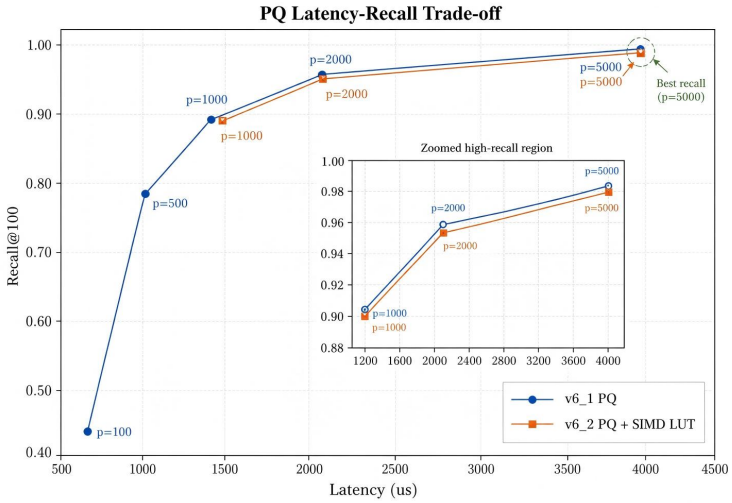


图 9: PQ 路线的 latency-recall 权衡关系

图 10 进一步比较了 v6_1 与 v6_2 在相同 rerank_p 下的延迟。可以看到，LUT SIMD 化带来小幅收益，但提升有限，说明 PQ 查询的主要开销仍在 ADC 全量扫描、候选维护和 rerank 阶段，而不是 LUT 构建本身。

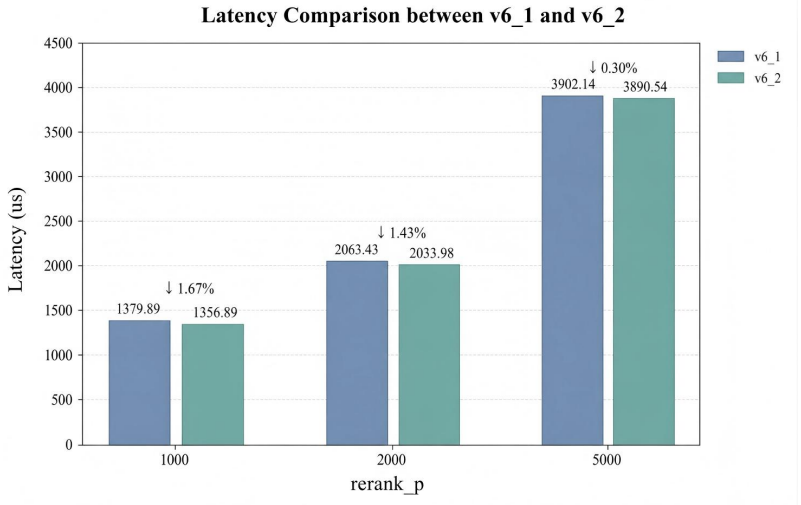


图 10: v6_1 与 v6_2 的同 rerank_p 延迟对比

4 Profiling 与汇编分析

4.1 perf stat 与 cache 分析

前一章主要从 average latency 和 recall@100 的角度比较了不同版本的整体表现。为了进一步解释性能差异的底层原因，本文使用 perf stat 对代表版本进行硬件事件统计。需要说明的是，本节中的 latency 来自 perf stat 运行时程序自身输出的单次 average latency[2]，主要用于与 cycles、instructions、IPC 和 branch-misses 等指标配合分析。前文性能测试中的主结果则采用统一脚本下多次复测的平均值。因此，二者数值可能存在一定差异，但不影响本节对性能趋势的解释。

表 2: 代表版本的 perf stat 结果

版本	latency(us)	cycles	instructions	IPC	branch-misses
v0	16551.90	85.13B	118.27B	1.39	23.78M
v1	6107.79	31.49B	39.07B	1.24	0.84M
v4	5169.54	26.59B	25.87B	0.97	0.83M
v5_3_p100	6699.87	34.59B	119.89B	3.47	11.28M
v5_4_p100	3995.60	20.70B	31.49B	1.52	17.11M
v5_5_p100	4503.52	23.41B	33.55B	1.43	157.42M

从表 2 可以看到，v1 相比 v0 的 cycles 从 85.13B 降低到 31.49B，instructions 也从 118.27B 降低到 39.07B。这说明基础 NEON SIMD 显著减少了原始 float 内积中的指令数量和执行周期，是 Flat-SIMD 路线中最主要的单步收益来源。

v4 的 cycles 进一步降低到 26.59B，低于 v1。这与前文性能测试中 v4 优于 v1 的结果一致，说明循环展开、多累加器和 prefetch 等优化不仅降低了平均延迟，也反映为更少的总执行周期。v4 的

[2]由于 perf stat 会引入额外监测开销，且运行时集群负载可能变化，本节 latency 仅用于与 cycles、instructions、IPC 等硬件事件配合分析；最终性能结论以前文统一脚本下多次复测的平均结果为准。

branch-misses 与 v1 接近，说明其性能提升主要来自计算和访存调度优化，而不是候选维护逻辑的变化。

对比 v5_3_p100 和 v5_4_p100 可以更清楚地看到 SQ 粗排 SIMD 化的效果。v5_3_p100 使用标量 uint8 粗排，instructions 高达 119.89B，cycles 为 34.59B；而 v5_4_p100 将 uint8 dot product 改为 NEON SIMD 后，instructions 降至 31.49B，cycles 降至 20.70B。该结果说明 v5_4 的性能提升并非来自简单调整参数，而是因为粗排阶段的主要计算被有效向量化。

v5_5_p100 的 latency 和 cycles 均高于 v5_4_p100，同时 branch-misses 达到 157.42M，远高于其他版本。这说明 nth_element 虽然在算法复杂度上看似可能减少候选维护开销，但实际运行中引入了更多不规则分支和数据访问，导致分支预测失败显著增加。因此，v5_5 没有超过使用堆维护 top-p 的 v5_4，成为一个有价值的负向消融结果。

综上，perf stat 结果与前文性能测试结论相互印证：Flat-SIMD 降低了原始距离计算的 cycles，SQ-SIMD 进一步降低了粗排阶段的指令和周期，而 nth_element 的分支开销解释了其性能不如 v5_4_p100 的原因。

为进一步从访存角度解释不同版本的性能差异，本文补充采集了 cache-references、cache-misses、L1-dcache-loads 和 L1-dcache-load-misses 等事件。

表 3: 代表版本 cache 行为对比

版本	cache-references	cache-misses	cache miss rate	L1 miss rate
v0	38.83B	502.97M	1.30%	1.30%
v4	11.23B	254.94M	2.27%	2.27%
v5_3_p100	39.26B	141.91M	0.36%	0.36%
v5_4_p100	3.42B	126.06M	3.68%	3.68%
v5_5_p100	5.49B	145.64M	2.65%	2.65%

表 3 从访存角度补充解释了不同版本的性能差异。v0 的 cache-references 达到 38.83B，说明原始标量 Flat Scan 在完整扫描 float base 向量时产生了较大的访存访问规模。v4 引入 SIMD、循环展开和 prefetch 后，cache-references 降至 11.23B，说明优化后的 Flat-SIMD 不仅降低了计算指令数量，也减少了运行过程中的数据访问压力。

v5_3_p100 的 cache miss rate 仅为 0.36%，但其 cache-references 仍达到 39.26B，说明标量 SQ 粗排虽然具有较好的局部性，但总体访问规模仍然较大。相比之下，v5_4_p100 的 cache-references 降至 3.42B，表明 uint8 SIMD 粗排显著改变了主扫描阶段的数据访问模式，并减少了需要处理的数据访问量。虽然其 cache miss rate 相对较高，但由于总访问次数大幅下降，整体 latency 仍低于 v5_3_p100。

此外，v5_5_p100 的 cache-references 和 cache-misses 均高于 v5_4_p100，结合前文 branch-misses 明显升高的现象，可以说明 nth_element 带来的不规则分区访问和条件分支可能削弱了候选选择阶段的实际性能表现。因此，cache 事件进一步支持了本文选择 v5_4_p100 作为最终主版本的结论。

4.2 汇编指令验证

为了确认源码中的 SIMD 和 prefetch 优化确实被编译为底层 ARM 指令，本文进一步使用 g++ -O2 -S 生成汇编文件，并检索关键指令。表 4 总结了不同版本中观察到的代表性汇编指令及其对应的算法模块。

表 4: 关键汇编指令及其对应优化

版本	关键指令	对应优化	说明
v4	prfm	prefetch	预取后续 base 向量数据
v4	fmla / fadd	float SIMD 内积	原始 float32 内积被映射为 NEON 向量乘加与累加
v5_4_p100	umull	uint8 SQ 粗排	量化向量粗排使用整数向量扩宽乘法
v5_4_p100	addv	SIMD 横向规约	将 SIMD 累加结果规约为标量 score
v5_4_p100	fmla / fadd	float rerank	候选精排阶段复用浮点 SIMD 内积
v6_2	fmla / faddp	PQ LUT SIMD	query LUT 构建中的短向量内积被 SIMD 化

对于 v4，汇编中观察到 prfm 指令，说明源码中的 __builtin_prefetch 被编译为 ARM 平台的预取指令。同时，fmla 和 fadd 的出现说明原始 float 内积确实被映射为 NEON 浮点向量乘加和向量累加操作。这与 v4 在 Flat-SIMD 路线中取得最低延迟的结果一致。

对于最终版本 v5_4_p100，汇编中出现了 umull 和 addv。其中，umull 表示对 8-bit 数据进行扩宽乘法，符合 SQ 粗排阶段对 uint8 量化向量进行乘加计算的实现逻辑；addv 用于将向量累加结果规约为标量分数。此外，fmla 和 fadd 仍然存在，说明候选 rerank 阶段继续使用了 float SIMD 内积。

对于 v6_2，汇编中观察到 fmla 和 faddp，对应 PQ 路线中 query LUT 构建阶段的 SIMD 内积与规约操作。该结果说明 v6_2 的 LUT SIMD 优化确实进入了底层指令，但由于 LUT 构建规模远小于 ADC 全量扫描和候选维护阶段，因此其整体收益较小。

因此，汇编验证从指令层面支撑了前文的实验结论：v4 的 prefetch 和 float SIMD 内积、v5_4 的 uint8 SQ SIMD 粗排，以及 v6_2 的 LUT SIMD 优化均已实际生效。

4.3 自动向量化与手写 SIMD 对比

除了手写 NEON SIMD，本文还测试了编译器自动向量化版本，用于分析仅依赖编译器优化普通 C++ 循环是否能够达到类似性能。图 11 给出了 ARM 平台上自动向量化版本与手写 SIMD 版本的平均延迟对比。

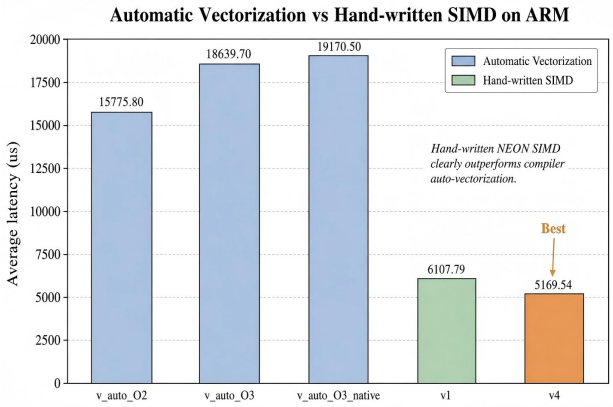


图 11: ARM 平台自动向量化与手写 SIMD 的平均延迟对比

从图 11 可以看到，v_auto_O2、v_auto_O3 和 v_auto_O3_native 的延迟均明显高于手写 SIMD 版本。其中，v_auto_O2 的 latency 为 15775.80 μs，接近原始 baseline；而手写 NEON SIMD 的 v1 已降至 6107.79 μs，v4 进一步降至 5169.54 μs。这说明在本实验的 ANN 全量内积扫描中，编译器自动向量化无法替代针对算法结构手工设计的 SIMD 实现。

值得注意的是，v_auto_O3 和 v_auto_O3_native 的 instructions 明显少于 v_auto_O2，但 latency 和 cycles 反而更高，IPC 也从 1.45 降至约 0.75。这说明指令数减少并不必然带来性能提升。更激进

的编译优化可能引入更长的依赖链、更高的寄存器压力或不利于流水线调度的代码结构，导致实际执行周期增加。

为进一步解释这种差异，本文对自动向量化版本和手写 SIMD 版本的汇编结果进行了对比，主要观察如下：

- **自动向量化版本：**虽然部分循环能够生成向量指令，但整体向量化并不稳定，仍包含较多控制逻辑和额外数据搬移，说明编译器会受到指针别名、循环边界、规约依赖和 top-k 维护等因素限制。
- **`v4` 手写 SIMD：**汇编中观察到 `fmla`、`fadd` 和 `prfm` 等指令，分别对应 float SIMD 乘加、累加和 prefetch，说明手写版本能够显式控制内积计算和访存预取。
- **`v5_4_p100` 手写 SIMD：**汇编中观察到 `umull` 和 `addv` 等指令，分别对应 uint8 粗排中的扩宽乘法和向量规约，说明 SQ 粗排也被有效映射到底层整数 SIMD 指令。

因此，手写 SIMD 的优势不仅在于能够生成 SIMD 指令，还在于可以围绕具体数据布局、向量宽度和规约模式显式设计内层循环。相比自动向量化，手写 NEON SIMD 能更稳定地控制向量加载、乘加、累加器数量、水平规约和 prefetch 策略。这解释了为什么在本实验的 96 维 ANN 内积扫描中，手写 SIMD 版本明显优于编译器自动向量化版本。

4.4 ARM 与 x86 自动向量化对比

为了进一步分析平台差异，本文在本地 x86 WSL 环境中使用相同 DEEP100K 数据集测试了普通 C++ 自动向量化版本，并与 ARM 平台自动向量化结果进行对比。结果如图 12 所示。

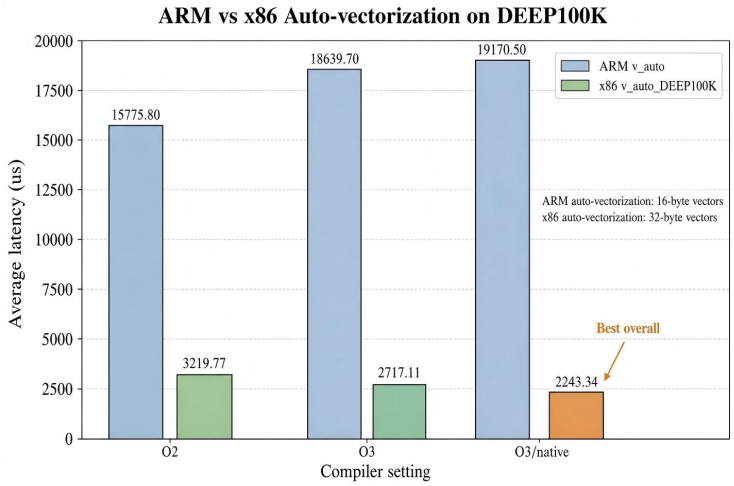


图 12: ARM 与 x86 平台自动向量化对比

从图 12 可以看到，在同样使用普通 C++ 自动向量化思路时，x86 平台的延迟明显低于 ARM 平台。其中，x86 上 `-O3 -march=native` 版本的平均延迟最低，为 2243.34 μs；而 ARM 上的自动向量化版本延迟仍在 15 ms 以上。

ARM 自动向量化主要使用 16-byte vectors，而 x86 的 `-O3 -march=native` 版本生成了 32-byte vectors；汇编中也观察到 `ymm` 寄存器和 `vmulps` 等 AVX 相关指令。这说明 x86 平台在该自动向量化实验中利用了 256-bit SIMD 宽度，而 ARM NEON 为 128-bit SIMD。SIMD 宽度、编译器后端、微结构调度和内存系统差异共同影响了最终性能。

5 结果分析与讨论

5.1 实验分析总结

综合性能测试、`perf stat` 与汇编验证结果，本文发现 ANN 查询优化并不存在单一最优实现，而是取决于 recall 目标：当目标是保持 recall@100 接近 1.0 时，`v5_4_p100` 是最优版本；当应用允许 recall@100 处于 0.9–0.95 区间时，PQ-ADC + rerank 路线能够以一定召回损失换取更低延迟。

表 5: 代表性方法综合对比

版本	方法类型	平均延迟/μs	recall@100	加速比
`v0`	Flat baseline	15955.27	0.99995	1.00×
`v4`	Flat-SIMD	5214.38	0.99995	3.06×
`v5_3_p100`	SQ scalar + rerank	6514.40	0.99995	2.45×
`v5_4_p100`	SQ-SIMD + rerank	3675.09	0.99995	4.34×
`v6_1_p1000`	PQ-ADC + rerank	1379.89	0.919541	11.64×
`v6_2_p2000`	PQ-ADC + LUT-SIMD	2033.98	0.956053	7.89×
`v6_2_p5000`	PQ-ADC + LUT-SIMD	3890.54	0.990501	4.13×

从表 5 可以看出，Flat-SIMD 路线能够在不损失 recall 的情况下显著降低精确扫描开销，其中 `v4` 将延迟从 `v0` 的 15955.27 μs 降至 5214.38 μs。在此基础上，`v5_4_p100` 通过 corrected SQ 粗排、uint8 NEON SIMD 粗排和 float SIMD 精排，将延迟进一步降至 3675.09 μs，同时保持 `recall@100=0.99995`，因此是高 recall 目标下的最佳综合方案。

PQ-ADC 路线则体现了另一类 latency–recall 取舍。`v6_1_p1000` 能够以 1379.89 μs 的延迟达到 `recall@100=0.919541`；`v6_2_p2000` 在 `recall@100=0.956053` 时延迟为 2033.98 μs，明显低于 `v5_4_p100`。因此，在 recall@100 接近 1.0 的高精度场景下，本文选择 `v5_4_p100`；而在 recall@100 为 0.9–0.95 已可接受的实用场景下，PQ-ADC + rerank 是更低延迟的选择。

负向消融结果进一步验证了上述选择。`v3` 的水平求和优化没有超过 `v2`，说明主要瓶颈不在末尾规约；`v5_1` 的 naive uint8 dot 使 recall@100 降至 0.10015，说明 SQ 粗排必须加入量化偏移修正；`v5_5` 的 `nth_element` 没有超过 `v5_4`，结合 branch-misses 激增可知，不规则分支和访存可能抵消理论上的候选选择优势。

5.2 实验局限讨论

首先，本文未进一步实现手写 AVX2/AVX-512，但主要是因为最终提交平台和主要性能评测平台均为 ARM aarch64，实验重点是 ARM NEON 手写 SIMD 优化；x86 WSL 仅作为自动向量化补充实验，用于观察不同 ISA 后端下编译器生成向量代码的差异。

此外，由于主实验运行在共享 ARM 集群环境中，单次 latency 可能受负载波动影响，因此主性能结论采用多次复测平均值，`perf stat` 结果主要用于解释硬件事件趋势。

本文还尝试了 OPQ 和 OpenMP，但二者均未作为最终主线。OPQ 分支在当前训练配置下召回率较低。OpenMP 分支虽然可能通过线程级并行进一步降低延迟，但其优化来源与本文重点讨论的 SIMD 指令级并行不同，且会引入线程调度和候选合并等额外变量。